



COMSM0129: Augmenting the Real World

Augmented Reality Cocktail Menu

Coursework Technical Report

Group B

Full Name	Student ID
Pranavv Jothinathan	2674319
Zhihan Yu	2718480
Yifei Liu	2637548
Skyler Sun	2750996

Table of Contents

1	 INTRODUCTION	3
2	 BACKGROUND.....	3
2.1	TRACKING	3
2.2	RAYCAST	4
2.3	LIGHT.....	4
2.4	3D MODELLING	4
3	 PROJECT IMPLEMENTATION	5
3.1	DESIGN PROCESS	5
3.1.1	Inputs and Outputs.....	5
3.1.2	Design Principles	6
3.2	IMPLEMENTATION PROCESS	6
3.2.1	GameObject Structure	6
3.2.2	Tracking Operation	6
3.2.3	Raycast Operation - Long Press and Short Press	8
3.2.4	Swipe Interaction to Change Models	12
3.2.5	View Mode for Users to Explore the Model in 360 degrees	14
3.2.6	Lighting.....	19
3.2.7	3D modelling	21
4	 CRITICAL EVALUATION.....	23
5	 CONCLUSION.....	24
5.1	TECHNICAL SUMMARY AND KEY TAKEAWAYS	24
5.2	DEVELOPMENT AND DEBUGGING SKILLS.....	24
5.3	FUTURE WORK AND EXTENSIONS	24
6	 REFERENCES	25

1 | Introduction

Traditional cocktail menus are still mostly text based. Customers need to imagine the drink based on the name and descriptions on menu when ordering. However, it's difficult to create a clear imagination only from text. This is especially true for cocktails with complex names or unique recipes. Customers often struggle to determine the color, glass shape, and overall style. So, Customers usually tend to choose familiar drink rather than trying a new one. This not only negatively impacts the customer's ordering experience, but also reduce the bar's effectiveness in promoting its signature cocktails.

To solve this issue, many bars have begun using picture cocktail menus. However, picture cocktail menus still have limitations. Images can only show the effect from a fixed angle. What is more, images cannot fully represent the state of a cocktail. For example, glass reflections, liquid layers, garnish details, and lighting are all difficult to demonstrate in 2D images. The drink that customers actually see often differs from the picture on the menu. This gap between expectation and reality can affect customer satisfaction and may reduce their willingness to visit the bar again.

In addition, cocktails are a type of drink that relies heavily on visual presentation. The appeal of many cocktails comes from their visual appeal and ambiance. However, traditional menus lack interactivity. Customers can only see text information and cannot actively explore the drinks. This approach makes it difficult for customers to experience the unique drinks. What is more, it limits the bar's performance in marketing and customer engagement. From a business perspective, if customers cannot understand the product's value, bars will find it harder to increase sales of drinks, thus impacting overall revenue

To solve these issues, this project designed and implemented an augmented reality cocktail menu system based on Unity AR Foundation. Compared to traditional menus, this system can directly place 3D cocktail models into the real environment. Users simply scan a coaster mark, and the system will generate the corresponding virtual cocktail on the table in real time.

The system supports multiple interaction methods. Users can switch between different beverages by swiping. Users can also view detailed information by clicking and long pressing. We additionally provide View Mode to freely observe model details from different angles. This approach helps users understand the details of cocktail. Our AR cocktail menu can enhance user's engagement during the ordering process. In addition, this project focuses on the integration of virtual objects with the real environment. The system implements dynamic lighting, environmental reflections, and real-time shadows. It allows virtual cocktails to adjust their visual appearance based on the real environment. Rather than simply floating on the screen, it is an augmented reality menu.

This project combines AR tracking, gesture interaction, lighting adaptation, and optimised 3D models. We aim to provide a more immersive and realistic ordering experience and helping bars and restaurants increase user interest and willingness to spend.

2 | Background

2.1 Tracking

The project focuses on establishing a connection between the physical world and the digital cocktail models. The tracking phase serves as the point where the connection begins. The project uses AR Image Tracking in AR Foundation for this purpose. A bottle coaster is used as the marker here. Once the user uses the device's camera

to show the coaster, the application would start to recognize the coaster. And as soon as the coaster recognizes the coaster, the application would analyze the visual features that are available and by using the coaster's position as a reference point, the application would anchor the 3-Dimensional cocktail models on top of the coaster in real time. This includes the models getting placed in a position, orientation relative to the tracked coaster.

2.2 Raycast

Following the successful detection of the target image and the completion of spatial registration for the virtual menu model, raycasting is used in this project to detect screen touch events to ensure that relevant information is displayed appropriately. These touch events are converted into virtual rays cast from the camera, which are then detected based on whether they hit the model's collider or layer button. This enables user to interact with the virtual cocktail object through gestures such as short tap, long press, and swipe, allowing cocktail details to be displayed. Additionally, this effectively prevents information overload on the mobile screen by default displaying only basic information.

Furthermore, to further enhance the user experience, multi-sensory feedback is provided following screen interactions with the object (Hughes CL and Karwowski W, 2025)^[1].

2.3 Light

This light part builds upon the techniques learned in Lab 7 of the unit, particularly the implementation of ambient light brightness and ambient color temperature^[2]. We also learned from Unity's AR Foundation, using the LightEstimation script and documentation to get how to return and deal with light data^[3].

For shadow implementation, we initially researched several existing solutions. It includes several free AR shadow tools^[4]. However, we found that these free tools often lacked the full functionality required for our specific ideas. Ultimately, we developed our own implementation, using an AR Shadow Shader sourced from additional technical resources. We implemented the transparent plane which can correctly render and receive shadows cast by the virtual models^[5].

For environmental reflection, we studied community implementations in YouTube. And we also studied official examples and documentations provided by Unity AR Foundation^{[6][7]}. These resources guided our use of Environment Probes to simulate the material properties of glass.

What is more, the idea of adding virtual light direction tracking came from Suhang. We were inspired to complete this additional feature.

2.4 3D modelling

In the early stages of the project, we conducted in-depth technical research on the presentation of the core interactive asset—the wine glass. To achieve the virtual liquid filling effect without triggering high-load flow calculations, we focused on the rendering scheme based on Stencil Shader^[8].

To obtain highly realistic environmental assets, we investigated photogrammetry. Preliminary research revealed significant limitations in processing non-Lambertian surfaces such as glass and mirrors. The reflective and warning properties of glass can lead to mismatches in feature point matching, resulting in geometric distortion. While the research mentioned using matte spray or transparent tape to mitigate unexpected failures, the reconstruction accuracy remains highly uncertain. Considering the potential failure risk of scanning glass, we considered scanning the base as an alternative.

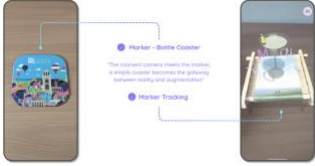
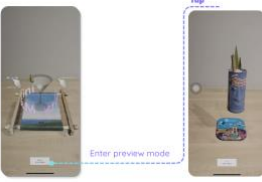
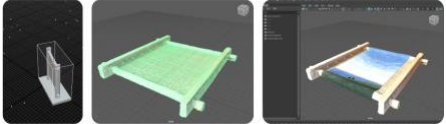
3 | Project Implementation

3.1 Design Process

3.1.1 Inputs and Outputs

User input is achieved through scanning and touching the screen, including scanning the image marker, long presses, short presses, swiping the screen, and clicking the button. And the corresponding output presents different user interfaces and provides feedback such as vibration and light. The main features are shown in Table 1:

Table 1. Feature List

Features	User Input	System Output	Prototype
Tracking	Scanning a marker	Generating a cocktail menu	
Raycast	Long Press	Popping up Basic Info UI Canvas Vibration Feedback	
	Short Press	Popping up Ingredient Info Canvas	
	Swiping	Switching between three different cocktails	
	Clicking View Mode button	Offering a 360-degree exploration of details	
Lighting	NA	Generating shadow on real plane Changing projection direction according to the real lighting	
3D asset modelling	NA	Virtual 3D base model	

3.1.2 Design Principles

Hassenzahl (2010)^[1] highlights that positive UXs are not solely determined by technical functionality but also by how the user feels and engages with the system. In AR, where engagement occurs in spatial, embodied contexts, superior UX requires more than efficient interfaces, it must foster presence, comfort, and emotional resonance to create seamless, empowering experiences that leverage the 3D nature of the environment (Hughes CL and Karwowski W, 2025)^[9]. According to the above definition of user experiences, this project defines design guidelines across two dimensions: usability and overall satisfaction.

- i. Usability: Evaluate whether a system is easy to learn and use from a functional perspective. This includes responding to user actions in real time and ensuring that gesture design follows Android's native design guidelines.
- ii. Overall satisfaction: Evaluate whether expectations are met from an emotional perspective, including excellent visual effects and multisensory feedback.

3.2 Implementation Process

3.2.1 GameObject Structure

Figure 1 shows the GameObject Structure of this project scene:

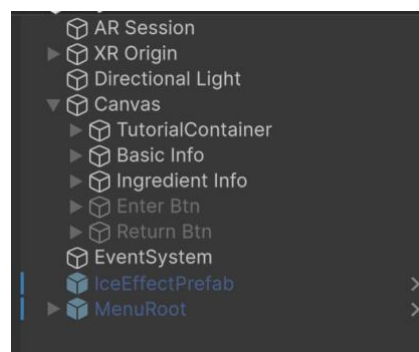


Figure 1. Hierarchy of AR Cocktail Menu

- i. AR Session enables and controls the lifecycle of all AR features, and XR Origin enables device tracking and transforms trackables into Unity's coordinate system. They are the standard GameObjects in the AR Foundation project.
- ii. Directional Light provides basic ambient lighting.
- iii. Canvas contains all UI elements on screen, and is used to display Tutorial, Basic Info, Ingredient Info and ViewMode button.
- iv. IceEffect is used to display the effects of the ice block falling and floating.
- v. MenuRoot is the root hierarchy for all 3D content in this project, containing the base, cloth, and cocktail models.

3.2.2 Tracking Operation

This section covers three main aspects: image tracking and virtual menu generation, opening animation management, and gesture guidance. And the core process and relevant control scripts are shown in Table 2 below:

Table 2. The Image Tracking Process and Control Scripts

Stage	Core Process	Control Script
1	Image tracking detection	ARTrackedImageManager ARImageSmoothFollow
2	MenuRoot is activated and parented to the trackedImage transform	ARImageSmoothFollow
3	MenuRoot child reference registration	CocktailARManager script
4	Opening animation starts	CocktailARManager script
5	Gesture tutorial appears	TutorialManager

Specifically, the **ARImageSmoothFollow** script is responsible for listening to image tracking, activating the MenuRoot and binding it to the tracked image; the **CocktailARManager** script handles the display and animation control of the MenuRoot; and the **TutorialManager** script controls the display, hiding and completion status of the gesture onboarding.

3.2.2.1 Image Tracking and MenuRoot Binding

The project initially used only the **ARTrackedImageManager** for image tracking. However, because the instantiated object (MenuRoot) in the project is a heavy prefab, instantiating it after successful image tracking increases the wait time following the first successful detection. This causes jitter and delay in the first frame. To reduce visible latency and make image tracking smoother, a new script (**ARImageSmoothFollow**) is subsequently added. This script pre-instantiates and hides the object in `Awake()`, then listens for the image tracking event from **ARTrackedImageManager**. Once the **TrackingState** changes to 'tracking', it activates and binds the MenuRoot instance (El Shemy, 2022)^[10], aligning the menuRootInstance with the `localPosition` of the tracked image. This is also the approach recommended in the official AR Foundation Unity documentation (see Figure 2)^[11].

Tracked Image Prefab

The **ARTrackedImageManager** has a "Tracked Image Prefab" field; however, this is not intended for content. When an image is detected, ARFoundation creates a new **GameObject** to represent it.

If "Tracked Image Prefab" is `null`, then AR Foundation creates a **GameObject** with an **ARTrackedImage** component on it. However, if you want every tracked image to also include additional components, you can provide a Prefab for AR Foundation to instantiate for each detected image. In other words, the purpose of the Prefab field is to extend the default behavior of tracked images; it is not the recommended way to place content in the world.

If you would like to **instantiate** content at the pose of the detected image and have its pose updated automatically, then you should parent your content to the **ARTrackedImage**.

Figure 2. The Recommendation for Tracked Image Prefab from the Unity Manual

3.2.2.2 Menu Generation and Animation Management

MenuRoot hierarchy is as follows (see Figure 3), it consists of Base, Cloth, and three Drink models. Because the goal is to animate MenuRoot after tracking — base popping out, cloth appearing, and drink falling into place — rather than generating a static model. If using just the default tracked image prefab, only static objects can be generated, so this project extends on the "Adding 3D Motions" task from Lab 3 (A. Zhang, 2026)^[12]. The **Tracked Image Prefab** is set to "None", and a new **CocktailARManager** script is added to control the dynamic visual presentation of MenuRoot.

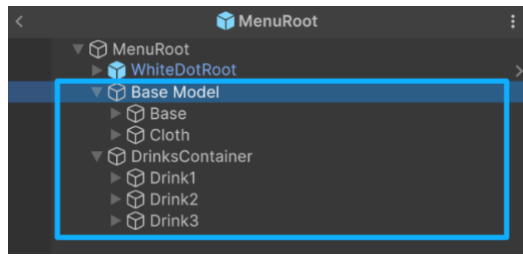


Figure 3. MenuRoot Hierarchy

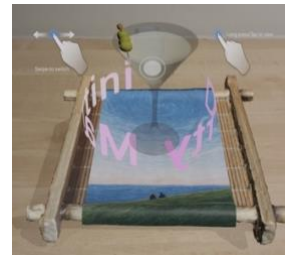


Figure 4. Gesture Onboarding

In particular, sequence animation effects are controlled by the **DOTween** plugin, which manages properties such as Transform and Material within the script, and uses Sequence to link the animations of each object. The end of Lab 3 instruction also recommends using the Unity Animation Manual as a reference for more complex 3D motion, but this may require additional state machines and event bindings. In contrast, **DOTween** offers a more lightweight approach [View introduction [here](#)].

To implement the opening animation, the coroutine requires integration with **DOTween**, the Coroutine is responsible for waiting for the animations to complete and proceeding to the next stage; **DOTween** is responsible for playing the animations for the base, the cloth, and the drink. Once the drink animation has finished, a delayed coroutine is used to play the particle effect corresponding to the current drink. The full timeline of the opening animation is as follows (See Table 3):

Table 3. Full Timeline of the Opening Animation

Time	Object	Event	Time	Object	Event
t = 0.0s	Base	• IntroSequence() starts • PlayBaseAnimation() is called	t = 1.5s – 2.2s	Cloth	• Fall back to clothOriginalLocalPos
t = 0.0s – 0.4s	Base	• Move upward to local Y = 0.15 • Scale from Vector3.zero to baseOriginalScale	t = 2.3s	Cloth & Drink	• IntroSequence() finishes waiting • ShowDrink(currentIndex) is called
t = 0.0s – 0.7s	Base	• Rotate 360 degrees around Y axis	t = 2.3s – 3.0s	Drink	• Move down from local Y = 0.25 to 0 • Scale from Vector3.zero to the original scale
t = 0.7s – 1.3s	Base	• Move back down to local Y = 0 • OutBack easing creates a landing bounce effect	t ≈ 3.0s	Drink & Particle Effect	• Drink DOTween animation completes • TriggerDrinkEffect(index)
t = 1.5s	Cloth	• Active via InsertCallback() • Place above the original position	t ≈ 3.5s	Particle Effect	• DelayedTriggerEffect() finishes 0.5s delay • Instantiate the current particle effect
t = 1.5s – 1.9s	Cloth	• Fade from transparent to opaque	t = 1.5s – 2.2s	Cloth	• Fall back to clothOriginalLocalPos

3.2.2.3 Tutorial Management

The gesture onboarding is designed to appear when the user first opens the app and has not touched the screen, and to disappear after any subsequent touch interaction. It consists of an indicating white dot, as well as visual guides for tap and swipe gestures, as shown in Figure 4.

3.2.3 Raycast Operation - Long Press and Short Press

Both long and short presses are controlled through the ComprehensivePressHandle script, the core process is as follows: detect screen touch input → convert screen touch into a raycast from AR Camera → identify the hit GameObject → classify the gesture as long press and short press → display the relevant UI Canvas.

3.2.3.1 Role of ComprehensivePressHandle script

The comprehensivePressHandle script is used to manage press-based interaction. The following presents the external data dependency, state variables, and flow diagram of this script.

i. External Data Dependency (as shown in Table 4)

Table 4 - External Data Dependency of the ComprehensivePressHandle script

Type	GameObject / Component	Purpose	Type	GameObject / Component	Purpose
Input	Input.GetTouch(0)	Detect screen touch	Data Source	BasicInfoData	Long press to cache Basic Info data
				IngredientData	Short press to cache Ingredient Info data
Camera	arCamera	Convert touch to ray	UI	basicInfo	Long press to display Basic Info Canvas
Physics	Physics.RaycastAll	Detect 3D Colliders		ingredientInfo	Short press to display Ingredient Info Canvas
Filter	interactableLayer	Limit detectable objects	Visual Feedback	RadarChartGrowth	Flavour radar animation
Target Type	IngredientData	Check for ingredient targets	Guidance	TutorialManager	Remove tutorial after completion
Target Type	cocktailTag = "Cocktail"	Check for cocktail shells			

ii. State Variables Explanation

- lastTouchedObject: Cache the object hit when the touch begins, as long press/short press is determined in subsequent frames.
- isTouching: Determine whether the current touch has hit a valid object.
- touchStartTime: Used to calculate the press duration.
- touchStartPos: Used to calculate the distance of the touch movement, to avoid mistaking a swipe for a press.
- hideCoroutine: Used to manage the auto-hide coroutine for the Basic Info Canvas.
- liquidHideCoroutine: Used to manage the auto-hide coroutine for the Ingredient Info Canvas.

iii. Flow Diagram (See Figure 5)

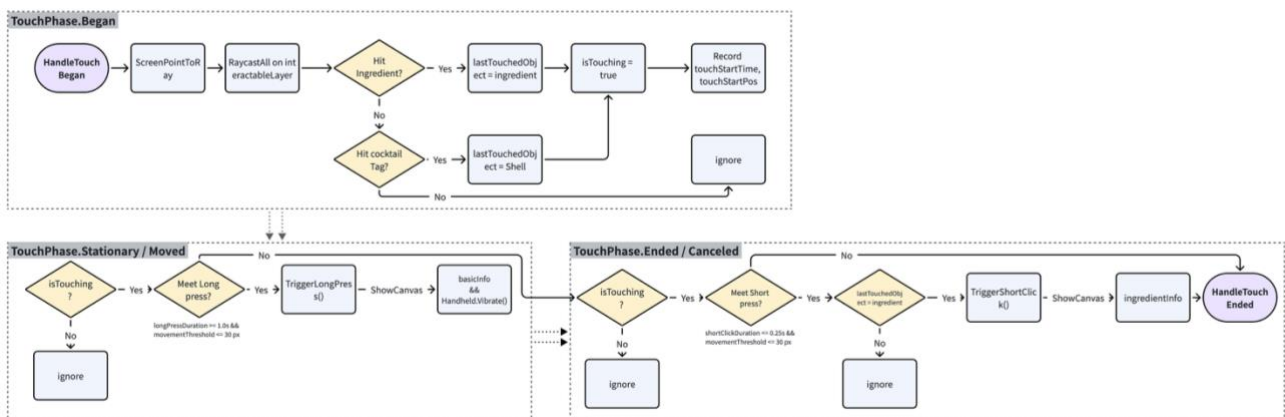


Figure 5. Workflow of ComprehensivePressHandle

3.2.3.2 Screen Touch Conversion and Object Hit Detection

This project follows the method used in Lab 5_Task 1 for “Implement Raycasting Logic” (A. Zhang, 2026)^[13], namely using **ScreenPointToRay** to convert screen touch input into a ray in the 3D scene. All drink GameObjects that support press interaction (including the glass and its contents) should set their **interactableLayer** to “AR_Cocktail”, and the **IngredientData** component should be mounted to the content GameObject. First, use **Physics.RaycastAll(ray, interactableLayer)** to check whether the ray has hit the GameObject in the **interactableLayer**, that is, the collider on the “AR_Cocktail” layer. Then, based on the existence of the **IngredientData** component, classify the type of object hit — if it exists, the hit represents a specific ingredient, and subsequent supported press gestures include both long press and short press; if it does not exist, the hit represents the glass shell, and only long press is supported.

3.2.3.3 Distinction between Long Press and Short Press

Listing 2 presents a method for distinguishing between long presses and short presses using **PressDuration** and **movementThreshold**. A long press is a hold where **longPressDuration** ≥ 1.0 s and **movementThreshold** ≤ 30 px; a short press is when the finger leaves the screen with **shortClickDuration** ≤ 0.25 s and **movementThreshold** < 30 px. The **movementThreshold** is added here to avoid conflicts with swipe interactions.

```
[SerializeField] private float longPressDuration = 1.0f;
[SerializeField] private float shortClickDuration = 0.25f;
[SerializeField] private float movementThreshold = 30f;
... ..
// Detect long press
if (isTouching && Time.time - touchStartTime >= longPressDuration && Vector2.Distance(touch.position, touchStartPos) <= movementThreshold)
{
    TriggerLongPress();
    isTouching = false;
}
// Detect short press
float touchDuration = Time.time - touchStartTime;
Vector2 swipeDelta = touch.position - touchStartPos;

if (isTouching && touchDuration <= shortClickDuration && swipeDelta.magnitude < movementThreshold)
{
    TriggerShortClick();
}
```

Listing 2. Long press and short press classification

3.2.3.4 Display of UI Information

Raycast should not only detect whether a hit occurs, but also provide visual feedback. As mentioned in Lab 5, this is an essential part of creating Interactive AR (A. Zhang, 2026)^[13]. Listing 3 shows the method for displaying Basic Info Canvas and generating a flavour radar after a long press. If a long press is detected, the **BasicInfoData** is fetched from the parent of the currently targeted GameObject, written to the Basic Info Canvas, and the radar animation is played via **RadarChatGrowth**. And the method for displaying Ingredient Info Canvas after a short press is similar to that for a long press. If a short press is detected, the **IngredientData** is fetched from the currently targeted GameObject, and the text content is written to the Ingredient Info Canvas. **IngredientData** and **BasicInfoData** are data components used to store the content displayed in the UI Canvas; they are mounted on the cocktail ingredients and the complete cocktail respectively (as shown in Figure 6).

```
private void TriggerLongPress()
{
    if (basicInfo != null && lastTouchedObject != null)
```

```

{
  #if !UNITY_EDITOR
    Handheld.Vibrate();
  #endif

  BasicInfoData data = lastTouchedObject.GetComponentInParent<BasicInfoData>();
  RadarChartGrowth growthEffect = basicInfo.GetComponentInChildren<RadarChartGrowth>();

  if (data != null && growthEffect != null)
  {
    if (nameText != null) nameText.text = data.cocktailName;

    if (descText != null) descText.text = data.cocktailDescription;

    TutorialManager.Instance.CompleteTutorial();

    HideCanvas(ingredientInfo, ref liquidHideCoroutine);

    ShowCanvas(basicInfo, ref hideCoroutine);
    growthEffect.AnimateRadarChart(data.GetValuesArray());
  }
}
}

```

Listing 3. Displaying Basic Info after long press

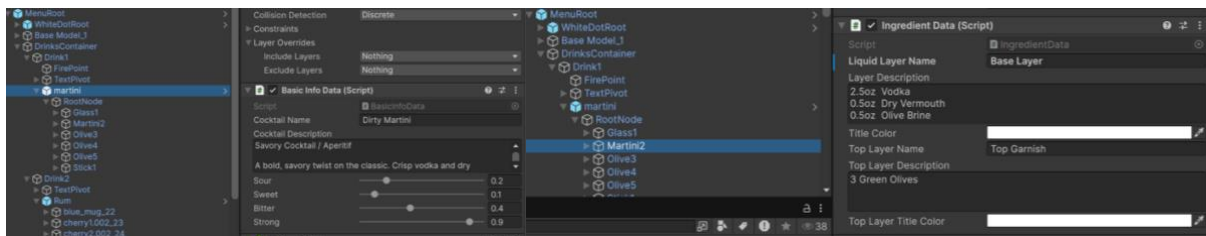


Figure 6. IngredientData is mounted on the drink's ingredient sub-hierarchy, whereas BasicInfoData is on the root hierarchy.

The structure of the Canvas GameObject is shown in Figure 7. It is used to define the display positions of the Image, TextMeshPro and Radar Chart. And the canvas uses ScreenSpace - Camera so that the UI is rendered by the AR Camera and displayed at a static position on the screen. Additionally, the Reference Resolution is set to 1080×1920 to align with mobile layouts.

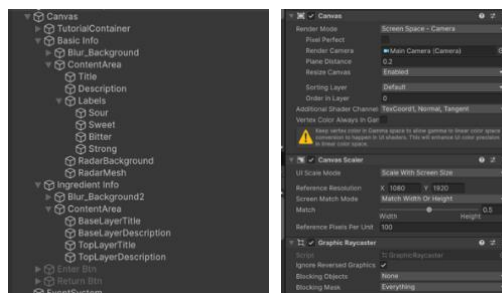


Figure 7. Canvas Structure, Render Mode, and Canvas Scaler

3.2.3.5 UI Canvas State Management

Basic Info and Ingredient Info employ an alternating display strategy, that is, when Basic Info is triggered by a long press, Ingredient Info is hidden first; when Ingredient Info is triggered by a short press, Basic Info is hidden first. This prevents the two information panels from being displayed simultaneously and overlapping. Furthermore, both Basic Info and Ingredient Info follow a logic of automatically hiding after three seconds of inactivity, consistent with the native Android Toast behaviour (Android Developers, 2026)^[17].

3.2.3.6 Vibration, Initial Onboarding and Animation Feedback

To enhance user engagement, a vibration feedback is triggered by **Handheld.Vibrate()** after a long press, and a dynamic radar chart is generated using **RadarChartGrowth.AnimateRadarChart()**. Additionally, the long-press trigger is marked as **TutorialManager.Instance.CompleteTutorial()** to remove the guidance display (See Listing 3 for the full code).

3.2.4 Swipe Interaction to Change Models

The purpose of the swipe interaction system in the application is to enable users to interact with the AR cocktail menu using the natural mobile device's touch gestures. The team chose swipe interaction as it is a common gesture used in mobile devices by the users for interaction and by using the same, it would make the users feel normal and the application would be easy to use. The swipe operation is overseen by the "Swipe_Script" script.

3.2.4.1 Core Process

The core process is as follows: User touches the screen → The touch start position would be stored → The user drags the finger horizontally and releases the finger → The touch end position would be stored → The swipe distance would be calculated and be compare with the set threshold value → If the distance is more than the set threshold value, then the direction would be identified → Based on the direction of the swipe, CocktailARManager script's either SwitchToPreviousDrink() function or SwitchToNextDrink() function would be called → The previous or next drink get be displayed on the tracked coaster.

3.2.4.2 State Variable Explanation

- touchStart – to store a touch's start position.
- touchEnd – to store a touch's end position.
- expectedWeightOfSwipe – to compare the swipe distance with the set threshold value of 50 units. Any swipe less than 50 units of distance would be discarded as accidental touches.

```
Using UnityEngine;

public class Swipe_Script : MonoBehaviour
{
    [SerializeField] private CocktailARManager arManager;
    public ViewMode_Script viewmodescript;

    private Vector2 touchStart;
    private Vector2 touchEnd;
    private float expectedWeightOfSwipe = 50f;

    void Start()
    {
        Debug.Log("Swipe Script Started");
    }
}
```

Listing 4. Variable Declaration and Reference Initialization

3.2.4.3 Workflow

Once the application anchors a digital model on top of the tracked coaster, the user would be able to change between models by swiping horizontally. The core operational part of the swipe system detects user touches in real time and determines if the swipes are non-accidental.

The operation starts by retrieving an active global instance of the CocktailARManager script. This is done because the CocktailARManager script is the core script that connects to other scripts.

```
void Update()
{
    arManager = CocktailARManager.Instance;
```

Listing 5. Instance Creation

i) Detecting Swipes

The system would check in real time if the user were touching the screen at any point by using 'Input.touchCount'. If it is '>0', then it means the user is touching the screen.

The first touch point by the user is retrieved using 'Input.GetTouch(0)' and the point of touch will be stored in 'touchStart'.

As the user drags the touch and releases the finger, the final touch point will also be stored in 'touchEnd'.

```
if (Input.touchCount > 0)
{
    Touch touch = Input.GetTouch(0);

    if (touch.phase == TouchPhase.Began)
    {
        touchStart = touch.position;
    }

    if (touch.phase == TouchPhase.Ended)
    {
        touchEnd = touch.position;
```

Listing 6. Touch Detection and Storage

ii) Swipe Distance Calculation and Validation

Once the start and end positions are obtained, the swipe distance would be calculated.

Formula: Touch End Position – Touch Start Position

'touchEnd.x - touchStart.x'

By subtracting the touch's end position from the touch's start position, the system calculates the swipe distance. The swipe distance would next be compared with the set threshold value of 50 units.

Using 'Mathf.Abs(currentWeightOfSwipe) > expectedWeightOfSwipe', any swipe more than 50 units of distance would move forward to the next step considering the swipe to be a valid and non-accidental swipe performed by the user with an intention of changing the current model.

```
float currentWeightOfSwipe = touchEnd.x - touchStart.x;

if (Mathf.Abs(currentWeightOfSwipe) > expectedWeightOfSwipe)
{
```

```
TutorialManager.Instance.CompleteTutorial();
```

Listing 7. Swipe Distance Comparison and Validation

iii) Swipe Direction and Model Switching

Swipe distance value → if Positive → Right swipe → Previous drink displayed

Swipe distance value → if Negative → Left swipe → Next drink displayed

Using 'currentWeightOfSwipe > 0', the system calculates if the swipe distance is positive. If it is a positive value, then the 'SwitchToPreviousDrink()' function in CocktailARManager script using its instance, would be called to switch to the previous drink. If it is a negative value, the 'SwitchToNextDrink()' function in CocktailARManager script using its instance, would be called to switch to the next drink. Based on the function, the respective drink model would be displayed.

```
arManager = CocktailARManager.Instance;

if (currentWeightOfSwipe > 0)
{
    arManager.SwitchToPreviousDrink(); // Swipe Right - Previous drink
}
else
{
    arManager.SwitchToNextDrink(); // Swipe Left - Next drink
}
}
}
}
}
```

Listing 8. Swipe Direction Calculation and Switching Models

3.2.5 View Mode for Users to Explore the Model in 360 degrees

The motivation to develop the 'View Mode' feature is to make the model available for the users explore it in any direction and angle the users likes, which might help the users decide their cocktail to have. While the normal display in the application allows the users to see the model from a fixed direction and angle, the view mode feature allows the users to change to any angle and direction.

Additionally, the View Mode feature would also strengthens the relationship between the real-time tracking and virtual objects models, by allowing the users to temporarily focus on the virtual model. This interaction feature also enhances the application by making it more engaging for the users rather than a simple Augmented Reality application.

3.2.5.1 Core Process

The core process is as follows: The user presses the View Mode button to enter → The currently displayed cocktail is retrieved → Swipe system is disabled → Animation and effects are disabled → Current cocktail is detached from the parent → Cocktail is placed near the camera for more visibility → User explores the model by rotating it → After exploration, the user presses the View Mode button to exit → The cocktail is re-attached to its parent → Swipe system is re-enabled → Animation and effects are re-enabled.

3.2.5.2 State Variable Explanation

- swipingScript – to create a reference for the swipe interaction script (Swipe_Script).
- arManager – to access CocktailARManager script as it controls the cocktail menu.
- buttonEnter – the button for users to enter the view mode.
- buttonExit – the button for users to exit the view mode.
- TargetGameObject – to retrieve the current cocktail model in display.
- inViewMode – a Boolean variable to check if the system is currently in view mode.
- initialParent – to store the original parent of the cocktail.
- initialLocalPos – to store the original position of the cocktail.
- initialLocalRot – to store the original rotation of the cocktail.

```
public class ViewMode_Script : MonoBehaviour
{
    public static ViewMode_Script Instance;
    public MonoBehaviour swipingScript;

    public CocktailARManager arManager;
    public GameObject buttonEnter;
    public GameObject buttonExit;

    private Transform targetGameObject;
    private bool inViewMode = false;

    private Transform initialParent;
    private Vector3 initialLocalPos;
    private Quaternion initialLocalRot;
}
```

Listing 9. Variable Declaration and Reference Initialization

3.2.5.3 Workflow

The system for view mode starts by resetting the interaction state and by initializing the user interface for the view mode. This is where the buttons to enter and exit the view mode are hidden during the application startup. Additionally, the view mode is made sure it is disabled during the application startup, by which the user can start with the swipe interaction without any disruptions or collisions.

```
void Start()
{
    if (buttonEnter != null) buttonEnter.SetActive(false);
    if (buttonExit != null) buttonExit.SetActive(false);
    inViewMode = false;
}
```

Listing 10. Resetting the Interaction State and Initializing View Mode User Interface

i) Monitoring Touch and converting to 3D Object Rotation

By using 'inViewMode && targetGameObject != null && Input.touchCount > 0', the entry criteria are set to ensure the rotation doesn't occur accidentally. This ensures the rotation occurs only inside the view mode, by ensuring there is a valid cocktail, and it also ensures the rotation occurs only when there is a touch by the user.

By using 'Input.GetTouch(0)', the system monitors for any touch by the user in real-time.

Rotation Calculation

To convert user's touch movement into rotational speed, 'touch.deltaPosition.x * 0.5f;' 'touch.deltaPosition.y * 0.5f;' are used.

'touch.deltaPosition.x * 0.5f;' is to calculate how much the model has to be rotated in Y-axis relative to the user's horizontal touch movement.

'touch.deltaPosition.y * 0.5f' is to calculate how much the model has to be rotated in X-axis relative to the user's vertical touch movement.

Applying the Rotation

'targetGameObject.Rotate(Vector3.up, -rotX, Space.World);' is used to rotate the model along the Y-axis which would result in left/right rotation of the model.

'targetGameObject.Rotate(Vector3.right, rotY, Space.World);' is used to rotate the model along the X-axis which would result in up/down rotation of the model.

```
void Update()
{
    if (inViewMode && targetGameObject != null && Input.touchCount > 0)
    {
        Touch touch = Input.GetTouch(0);

        if (touch.phase == TouchPhase.Moved)
        {
            float rotX = touch.deltaPosition.x * 0.5f;
            float rotY = touch.deltaPosition.y * 0.5f;

            targetGameObject.Rotate(Vector3.up, -rotX, Space.World);
            targetGameObject.Rotate(Vector3.right, rotY, Space.World);
        }
    }
}
```

Listing 11. Monitoring User Touch and Conversion of Touch into 3D Model Rotation

ii) User Entering the View Mode

When a user enters the view mode, there are a series of actions that take place. First, it starts by creating an active instance for the CocktailARManager as it manages the menu. The system prevents the user from entering the view mode when the model or the application is still animating.

```
public void EnterViewMode()
{
    arManager = CocktailARManager.Instance;

    if (arManager == null) return;
    if (arManager.IsAnimating) return;
```

Listing 12. Instance Creation and Preventing Entry to View Mode during Animation

The system starts the view operations by retrieving the current drink, using 'arManager.GetCurrentDrink()'. The system then clears off all the active effects running, by using 'arManager.ClearActiveEffect();'. The system hides the tutorial guidance using 'TutorialManager.Instance.TemporarilyHideGuidance(true);'.

```
Transform drink = arManager.GetCurrentDrink();
if (drink == null) return;

targetGameObject = drink;
arManager.ClearActiveEffect();

TutorialManager.Instance.TemporarilyHideGuidance(true);
```



```
}

```

Listing 13. Retrieval of Current Drink and Hiding Effects

All the DOTween animations are stopped by using 'targetGameObject.DOKill();'. By using 'targetGameObject.GetComponentInChildren<Rotator>();' and 'targetGameObject.GetComponentInChildren<Animator>();', the system disables any automatic rotation and animations respectively. To hide the AR base and cloth of the normal mode environment, 'baseModel.gameObject.SetActive(false);' and 'clothModel.gameObject.SetActive(false);' are used.

```
targetGameObject.DOKill();
Rotator rot = targetGameObject.GetComponentInChildren<Rotator>();
if (rot != null)
{
    rot.gameObject.SetActive(false);
}

Animator anim = targetGameObject.GetComponentInChildren<Animator>();
if (anim != null) anim.enabled = false;
if (arManager.baseModel != null) arManager.baseModel.gameObject.SetActive(false);
if (arManager.clothModel != null) arManager.clothModel.gameObject.SetActive(false);
```

Listing 14. Stopping Animations and Automatic Rotations.

The system then stores the original transform information of the current model like the parent, original position, and the original rotation. The system then disables the swipe interaction to avoid model switching when the application flow is inside the view mode. The AR manager is temporarily disabled to avoid any collisions due to updates in AR manager.

```
initialParent = drink.parent;
initialLocalPos = drink.localPosition;
initialLocalRot = drink.localRotation;

if (swipingScript != null) swipingScript.enabled = false;
arManager.enabled = false;
```

Listing 15. Storage of Original Parent, Position and Rotation

The button to enter the view mode would be disabled and the button to exit the view mode would be enabled. The current cocktail model is detached from its parent to place the model closer to the camera. The state of user inside the view mode would be set to a value of True. The camera feature is now grabbed and by using it, the cocktail model would be placed right in front of the camera for a much closer look, by using 'cameraTransform.position + cameraTransform.forward * 0.4f;'.

```
if (buttonEnter != null) buttonEnter.SetActive(false);
if (buttonExit != null) buttonExit.SetActive(true);

drink.SetParent(null);
inViewMode = true;

Camera camera = Camera.main;
if (camera == null) return;

Transform cameraTransform = camera.transform;

drink.position = cameraTransform.position + cameraTransform.forward * 0.4f;
drink.rotation = Quaternion.identity;
```

```

    Debug.Log("VIEW MODE ENABLED: " + drink.name);
}

```

Listing 16. User Interface Change and Model Placement

iii) User Exiting the View Mode

The purpose of exiting the view mode part is to make sure the application returns back to the way it was with all operations in place running smoothly just like how it was before entering the view mode. The flow of exiting the view mode starts by restoring the current cocktail model to its initial parent and restoring the model back to its initial position and rotation.

```

public void ExitViewMode()
{
    if (targetGameObject == null) return;

    targetGameObject.SetParent(initialParent);
    targetGameObject.localPosition = initialLocalPos;
    targetGameObject.localRotation = initialLocalRot;
}

```

Listing 17. Restoring Model back to its Initial Parent, Initial Position and Initial Rotation

The system then restores the tutorial guidance for the users by using 'TutorialManager.Instance.TemporarilyHideGuidance(false);'. To restore the normal automatic object behaviour, 'rot.enabled = true;' is used. To restore the normal AR scene updates, 'arManager.enabled = true;' is used. The system also restores the normal swipe interaction control for users to change to other models.

```

TutorialManager.Instance.TemporarilyHideGuidance(false);
Rotator rot = targetGameObject.GetComponentInChildren<Rotator>(true);
if (rot != null)
{
    rot.gameObject.SetActive(true);
    rot.enabled = true;
}
arManager.enabled = true;

if (swipingScript != null) swipingScript.enabled = true;

```

Listing 18. Re-enabling Tutorial Guidance, Restoring Normal Object Behaviour and Scene Updates

By retrieving the children components of the animator, and setting them to True, the system restores all the regular animations and its related updates. The environment models such as the base model and the cloth model are restored back to the scene by setting both the base model and cloth model to the value of True.

```

Animator anim = targetGameObject.GetComponentInChildren<Animator>();
if (anim != null) anim.enabled = true;

if (arManager.baseModel != null) arManager.baseModel.gameObject.SetActive(true);
if (arManager.clothModel != null) arManager.clothModel.gameObject.SetActive(true);

```

Listing 19. Restoring the Animations and the Base Model and Cloth Model

The button to enter the view mode would be enabled, and the button to exit the view mode would be disabled. The state of user inside the view mode would be set to False.

```

if (buttonExit != null) buttonExit.SetActive(false);
if (buttonEnter != null) buttonEnter.SetActive(true);
inViewMode = false;

```

```

Debug.Log("VIEW MODE EXITED");
}

```

Listing 20. User Interface Change

3.2.6 Lighting

3.2.6.1 Color and Brightness Optimization

Initially, we followed the script from the labs of unit to synchronize the virtual light's color with the real light condition. We found that color wasn't important for the realism of virtual objects. Because the ambient light in most environments was close to white. Users could barely feel changes about color temperature.

However, changing brightness is much more difficult. We observed that simply setting the virtual light brightness to the value returned by the AR camera is not enough. It still makes objects appear unrealistic. For example, when using a hand to cover the phone camera, the change in virtual brightness was not obvious enough to the user.

What is more, if the light is too bright or too dark, the virtual object will be very unrealistic. If it is too bright, making the user feel that it is glaring. If it is too dark, making the virtual object look like a blot of ink on the screen. To solve this problem, we denote the intensity returned by the AR camera in light estimation as x . The parameters for the dark and bright ends were selected through several tests. First, we use parameters a and b to linearly map the readings in this interval to $[0, 1]$. Then use function $\text{clamp}(0, 1)$ to limit the result to $[0, 1]$. It will be used for further brightness mapping.

$$t = \text{clamp}\left(\frac{x - a}{b - a}, 0, 1\right)$$

By applying a power function to this normalized data, we were able to make the light intensity change more rapidly in bright environments. It enabled the virtual model to remain visually dynamic.

$$t' = t^\gamma$$

And in dark environments, the power function could scale the intensity more slowly. This allows users to perceive slower changes in light in dark environments.

3.2.6.2 Environmental Reflection

Reflection is essential for simulating smooth materials such as glass. Based on Lecture 6 and Unity's AR Foundation documentation, we imported Environment Probes Manager. This component can capture the surrounding reality and map it onto our virtual models^{[6][15]}. When we followed the standard steps of adding an environment probe manager, we failed to see the reflection.

We suspected that a version update of Unity might require different component settings. To debug this issue, we first tried to place environment probes in the scene instead of automatically placing environment probes. But we still cannot see reflection. To determine whether the problem is the model or the lighting system, we replaced the virtual glass with a completely smooth sphere. We observed that the probe manager successfully showed Unity's default skybox. But in reality, the reflection was not clear like Unity default skybox.

To further investigate, we used the user interface to output the texture resolution values returned by the camera. We ultimately discovered that the environment cubemap returned by the camera had a resolution of only 32x32 pixels. Due to this extremely low resolution, reflections were blurry. We concluded that this lack of visual clarity is a hardware limitation of the mobile device itself. And it is not a problem with our software implementation.

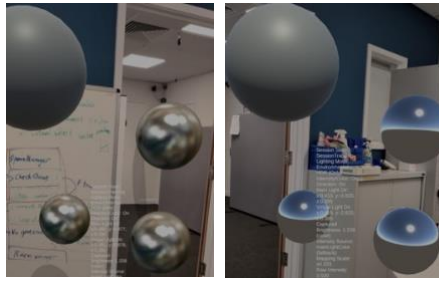


Figure 8. Left is with Environment Probe Manager. Right is without.

3.2.6.3 Shadow and AR plane

Shadows are essential for connecting virtual objects to real plane surfaces. We need to adjust shadow settings in the Unity editor and constantly observe their effects. Such as jagged edges and depth are the effects we need to observe. During development, we initially encountered a problem. That is shadow changes weren't displayed in real-time in the Unity editor. With Jon's technical guidance, we discovered that we needed to switch the graphics and quality options in the player settings from the PC renderer to the mobile renderer. Then we can see realtime changed of shadow.

The implementation logic is as follows. The AR plane manager is used to detect real-world surfaces, and then a transparent plane prefab is instantiated to receive shadows. However, we found that the default AR plane prefab contains visible colors and boundaries. This reduces the user's immersion. To address this, we accessed the line renderer component of the AR plane prefab and set the alpha value of its color to zero. This makes the plane boundaries transparent. For the color, we used shaders to implement the transparent plane and cast shadows.

Error!
Reference source not found.

Regarding shadow type, we chose hard shadows over soft shadows. Although soft shadows can reduce jagged edges, we found them ineffective at rendering the shape of a wine glass. Furthermore, to balance visual effect and project lag, we set the shadow quality to medium. Our tests showed that higher quality settings resulted in noticeable application lag. This was particularly noticeable when multiple models were present or when the user rotated virtual objects.

3.2.6.4 Light Direction

For lighting direction, we used the main light direction. However, we discovered a significant conflict. When both the main light source and ambient light estimation were enabled, both the ambient light intensity and ambient light color values returned "None." We verified this through the user interface output. The results show that the system only returns main light data when both are enabled.

The conflict arose because Unity uses two different methods for representing lighting, ambient light and environment light. Therefore, we decided against using the method from our previous labs. Then we focused on the main light data.

What is more, our testing found that the data of light direction does not change immediately. By monitoring the UI output, we found that the latency is about 2 seconds. This delay is a confirmed device limitation, rather than an implementation error.

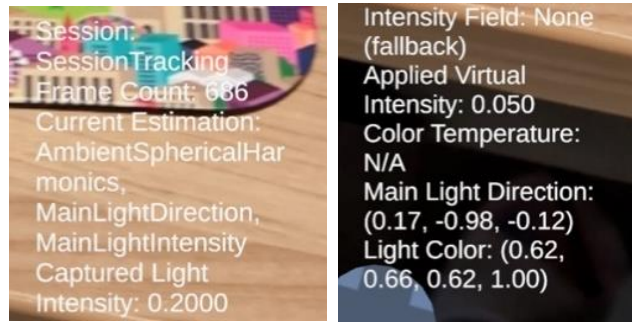


Figure 9. Output none for the ambient light intensity.

3.2.7 3D modelling

3.2.7.1 Model selection

In the initial stage of the project, our original plan was to use a glass cup as the core interactive asset. We attempted to collect data from it and then simulated liquid flow. As a result, we designed a liquid simulation scheme based on a Stencil Shader (See Figure 10). The core logic of this scheme is to use a Stencil Buffer as a mask, and by controlling the rendering order and stencil values, visually achieve the effect of liquid being "pollinated" into the glass and precisely filling it. This method avoids complex fluid calculations and can maintain a high frame rate on mobile devices. However, we encountered a challenge: Photogrammetry struggled with the glass's transparency and reflections, leading to a distorted model (See Figure 11).



Figure 10. Liquid simulation

To solve the problem, we tried coating the cup with flour to kill the reflections and create more trackable texture. But honestly, even with that, the mesh quality just wasn't meeting our standards for a 'high-quality model.'(See Figure 12) So, we decided to change to a bamboo base instead, which worked way better. (Xoio, 2021)^[16].

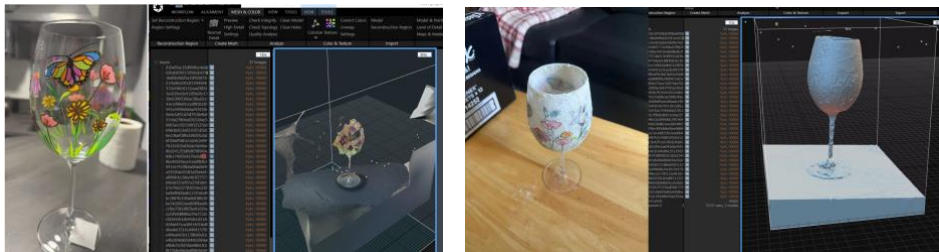


Figure 11. Original glass cup and dictorted model

Figure 12. Coating the cup with flour

3.2.7.2 Processing and mesh optimization

For the bamboo base, we captured high-resolution images from multiple dimensions under uniform lighting conditions to ensure the accuracy of the point cloud reconstruction. After that, we used RealityCapture software for point cloud alignment and generated a high-precision polygon mesh (See Figure 13). Considering the performance

limitations of mobile devices, we used the Simplify Tool to reduce the number of facets in the model. By setting the target triangle count to around 50,000, we significantly reduced the computational load while maintaining visual features (See Figure 14).

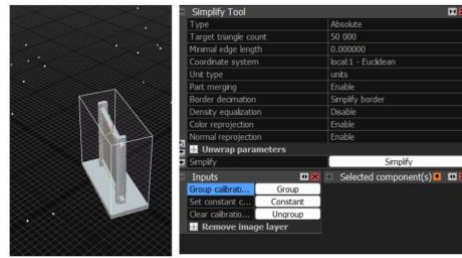
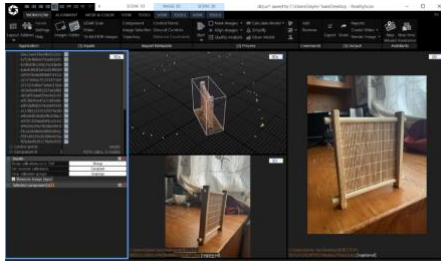


Figure 13. RealityCapture workflow Figure 14. Simplify Tool to reduce the number of facets in the model

In the Optimization phase, we imported the raw mesh into Maya for high-precision geometry cleaning. We utilized sculpting and smoothing tools to eliminate scan noise and artifacts. Finally, we performed coordinate system alignment on the model in Maya to ensure that it could be instantiated with the correct pose and scale in the Unity AR environment (See Figure 15).

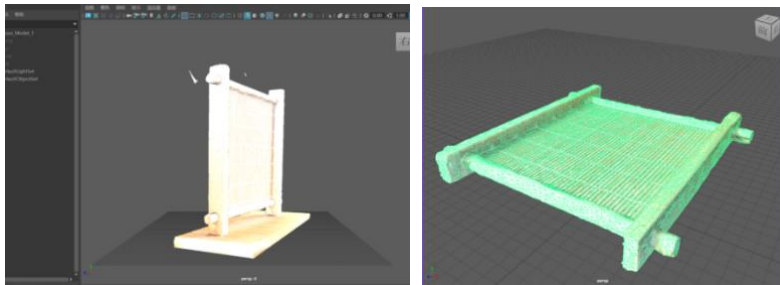


Figure 15. Optimization in maya

3.2.7.3 Integration with simulation

The optimized base model not only serves as a visual background but also as a physical collider in subsequent animation generation. In Maya, we used nCloth to simulate the physical dynamics of the tablecloth being laid out. Because the base model accurately reproduces the real geometry through height mapping, the folds and coverage of the tablecloth appear very natural, enhancing the immersive experience of the scene.

After importing into Unity, we applied a double-sided shader to the thin sheet models such as the tablecloth to ensure that the materials do not appear transparent or disappear from any angle when viewed by the user in an AR environment. We then combined this model with downloaded cocktail models (See Figure 16).

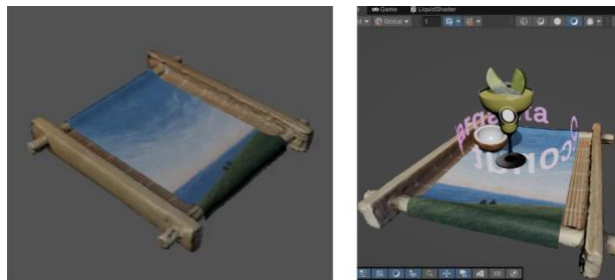


Figure 16. Integration models in Unity

4 | Critical Evaluation

After completing the project development, we conducted a user experience survey. Based on [3.1.2 Design Principles](#), we divided the survey into four dimensions: usability, visual quality, stability, and open-ended feedback [View the survey [here](#)]. The goal was to identify gaps in the user experience of the current project's features and further clarify the direction for future iterations.

The sample size for this user survey was ten. Overall, user satisfaction was high, and there was a strong willingness to recommend the app, with an average interest score of 4.8 out of 5. Users generally agreed that the AR Cocktail Menu is fun, visually appealing, and runs smoothly (See Figure 17). However, the results also indicated that there is room for improvement in the project's gesture interaction and model details.

In terms of usability, all participants reported being able to clearly view specific drink information and understood the swipe gesture for switching between drinks. This highlights the necessity of implementing gesture onboarding and confirms that the project's basic interaction logic aligns with users' operational habits. Although users indicated they knew how to operate the system, 40% still reported difficulty during actual use (See Figure 18). This provides a direction for future optimization: the issue of the "press" event being difficult to trigger can be addressed by expanding the clickable hotspots. Additionally, after entering view mode, the model's rotation range should be restricted to support only 360-degree horizontal rotation, with vertical pitch limited to $\pm 15^\circ$, to prevent unexpected flipping caused by excessive rotational freedom during swiping and to enhance model controllability.



Figure 17. Overall Satisfaction Feedback



Figure 18. Usability Feedback

Users were satisfied with the visual quality, particularly regarding the generation and transition animations of alcoholic beverages, with an average score of 4.9 out of 5. However, 20% of users felt the textures and details of the cocktail models still had room for improvement (See Figure 19). Additionally, in open-ended feedback, some users mentioned a desire to see more physically realistic liquid effects (See Figure 21). This indicates that future efforts should focus more on rendering the model's physical effects and restoring realism. Currently, the model has also received positive feedback regarding stability, with most users reporting no noticeable jitter, which validates the effectiveness of visible latency management (See Figure 20).

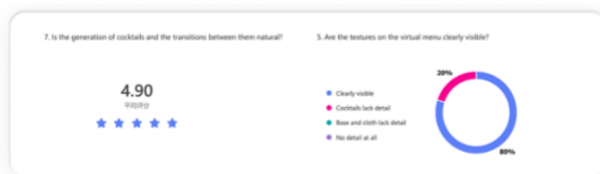


Figure 19. Visual Quality Feedback



Figure 20. Stability Feedback

The open-ended responses indicate that future iterations should focus on richer interactions and immersive experiences. Suggested improvements include liquid physics, cocktail mixing features, a wider selection of drinks, snack pairings, immersive audio, and voice assistants (See Figure 21). These responses suggest that users view this concept not merely as a visual menu, but as a potential interactive and personalised ordering experience.



Figure 21. Open-ended Feedback

5 | Conclusion

5.1 Technical Summary and Key Takeaways

This project, through the development of an interactive XR cocktail demo system, enabled the team to gain in-depth mastery of the four core pillars of AR development: tracking, raycasting, real-time lighting, and 3D modeling. In tracking and system integration, we not only achieved stable registration of virtual menus in the real-world environment but also effectively eliminated first-frame jitter and significantly improved overall tracking stability through pre-instantiation and parenting techniques.

At the asset creation and rendering level, we mastered the complete photogrammetry reconstruction workflow from physical objects to digital assets and learned performance-oriented asset optimization principles. By optimizing the mapping model to within 50,000 polygons, we ensured that the model maintained high visual fidelity while providing low-overhead physical colliders for raycasting, thus achieving a smooth interactive experience. Furthermore, by studying the underlying rendering pipeline, we learned how to achieve efficient liquid visual effects using Stencil Shader, and how to ensure lighting consistency between virtual assets and the real environment using the LightEstimation API and Environment Probes.

5.2 Development and Debugging Skills

Beyond the technical implementation, this project significantly enhanced our systematic debugging capabilities. We learned to solve rendering challenges unique to mobile devices, such as real-time shadows, by analyzing the differences in rendering paths between Unity's Mobile Renderer and PC Renderer. Simultaneously, the team became more proficient in consulting and applying Unity AR Foundation official documentation and community technical resources, enabling us to accurately determine whether problems stemmed from hardware limitations, engine behavior, or logical errors—crucial for future development of complex XR projects.

5.3 Future work and extensions

While the project is currently performing stably, there is still room for improvement in both the breadth and depth of its technical aspects:

1. Enhanced Tracking Stability: Plans include introducing filtering and prediction algorithms to further mitigate model jitter caused by sensor noise.
2. Improved Rendering Quality: Addressing the current limitations of reflection resolution (32x32 cubemaps), we aim to achieve high dynamic range reflections on higher-performance hardware and further optimize model mesh quality using manual retopology.
3. Intelligent Interactive Experience: Future goals include integrating an intelligent lighting control system, allowing users to customize the lighting atmosphere in real-time via voice or text commands (such as changing light intensity or hue), thereby providing a more immersive and personalized AR dining experience.

6 | References

- [1] Hughes CL and Karwowski W (2025) Evaluating interaction design and user experience in augmented reality: a systematic review. *Front. Virtual Real.* 6:1710161. doi: 10.3389/frvir.2025.1710161
- [2] A. Zhang, "COMSM0129_ARW: Source code for COMSM0129 ARW lab sessions," *GitHub repository*, 2026. [Online]. Available: https://github.com/fan-aaron-zhang/COMSM0129_ARW
- [3] Unity Technologies, "LightEstimation | AR Foundation | 6.0.7," *Unity Documentation*. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.0/api/UnityEngine.XR.ARFoundation.LightEstimation.html>
- [4] Dev Dunk Studio, "Shadow Receiver URP (FREE AR Shadows)," *Unity Asset Store*, 2026. [Online]. Available: <https://assetstore.unity.com/packages/vfx/shaders/shadow-receiver-urp-free-ar-shadows-228074>
- [5] D. Valecillos, "URP AR Shadow Receiver Shader," in *ARFoundationOcclusion*, feature/URPOcclusion branch, GitHub repository, 2022. [Online]. Available: <https://github.com/dilmerv/ARFoundationOcclusion/blob/feature/URPOcclusion/Assets/Shaders/URP%20AR%20Shadow%20Receiver.shader>
- [6] Unity Technologies, "Environment Probes | AR Foundation | 6.0.7," *Unity Documentation*. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.0/manual/features/environment-probes.html>
- [7] D. Valecillos, "Unity AR Foundation - How To Create Real World Reflections With AR Environment Probes?," *YouTube*, July 21, 2020. [Online Video]. Available: https://www.youtube.com/watch?v=5M_Hov08U48
- [8] Unity Technologies (2021) *ShaderLab: Stencil*. Unity User Manual. Available at: <https://docs.unity3d.com/cn/2021.1/Manual/SL-Stencil.html> (Accessed: 10 May 2026).
- [9] Hughes CL and Karwowski W (2025) Evaluating interaction design and user experience in augmented reality: a systematic review. *Front. Virtual Real.* 6:1710161. doi: 10.3389/frvir.2025.1710161
- [10] El Shemy, I. (2022, November 16). Image tracking with ARFoundation. Medium.
- [11] Unity Manual. (2026). AR Tracked Image Manager component. AR Foundation 6.0.7 documentation. <https://docs.unity3d.com/Packages/com.unity.xr.arfoundation%406.0/manual/features/image-tracking.html#tracked-image-prefab>
- [12] Zhang, A. (2026). *ARW_Lab_3*. [Lab notes] COMSM0129: Augmenting the Real World, University of Bristol. Available from: <https://www.ole.bris.ac.uk/>
- [13] Zhang, A. (2026). *ARW_Lab_5*. [Lab notes] COMSM0129: Augmenting the Real World, University of Bristol. Available from: <https://www.ole.bris.ac.uk/>
- [14] Hassenzahl M. (2010). *Experience design: technology for all the right reasons*. Springer International Publishing. 10.1007/978-3-031-02191-6
- [15] Zhang, A. (2026). *ARW_L6*. [Lecture notes] COMSM0129: Augmenting the Real World, University of Bristol. Available from: <https://www.ole.bris.ac.uk/>
- [16] Xoio (2021) How to photoscan Glass - and other Transparent and Glossy Objects. YouTube video. [Available at: https://www.youtube.com/watch?v=PF4Nc24I_x0](https://www.youtube.com/watch?v=PF4Nc24I_x0) (Accessed: 10 May 2026).
- [17] Android Developers. (2026, February 26). Toasts overview. <https://developer.android.com/guide/topics/ui/notifiers/toasts>.